

Climate and Health Metadata Catalogue — how it fits together

One source, two ways of reading it.

File	What it is	Who uses it
<code>catalogue.json</code>	The source. One object per dataset, one key per field.	You, when adding or editing a record
<code>climate_health_catalogue_register.csv</code>	The same content as a flat table. Opens in Excel or Google Sheets; import into CKAN or Airtable.	Partner agencies filling in their own rows
<code>climate_health_catalogue.html</code>	Single-file web page. Search, filter by project, domain and access, expand any record. No server and no build step. Works offline; only the two webfonts need a connection, and it falls back to system fonts without one.	Anyone who needs to look something up
<code>climate_health_catalogue.docx</code>	Printable and editable. One page per dataset, a summary table, the open gaps, the field definitions and a change log. Not in the repository — it embeds the gaps register, so it is built locally and shared directly.	The reviewer report, and anyone who prefers paper
<code>gaps.json</code>	The open gaps register — internal review material. Not in the repository. Kept beside the checkout; <code>build_docx.js</code> reads it if present.	The laboratory, when reviewing progress

The loop

Edit `catalogue.json`, then rebuild both outputs:

```
python3 build_web.py    # writes the CSV and the HTML
node build_docx.js      # writes the DOCX
```

The two PDFs are built from their Markdown with `convert2pdf` (a local script: python-markdown plus WeasyPrint, not in this repository):

```
convert2pdf README.md README.pdf
convert2pdf EDITING_GUIDE.md EDITING_GUIDE.pdf
```

`build_docx.js` also reads `gaps.json` if it is beside the checkout. Without it the DOCX builds without the gaps section — which is what anyone cloning the public repository gets, and is intended.

If you would rather work in a spreadsheet, edit the CSV and convert it back to JSON before rebuilding. Do not edit the DOCX and expect the change to survive a rebuild — copy it back into the source first.

Publishing the web page

The HTML is one file, so it travels anywhere — put it on the IPSP web server, or email it and it opens from disk. Its only external reference is the Google Fonts stylesheet for Montserrat and Noto Sans JP; offline it renders in the system sans fallback, with every colour, rule and layout intact.

It is also published as a GitHub Page:

<https://mauroccm.github.io/climate-health-catalogue/>

To update it, rebuild and push:

```
python3 build_web.py
git commit -am "Update catalogue"
git push
```

GitHub rebuilds on every push to `main`. The new version is live in under a minute.

Three things about that setup:

- **Only `docs/` is published as a page.** Pages serves that folder and nothing above it. `build_web.py` writes the page into `docs/` and the register into the repo root — that split is the whole mechanism, so do not move files between the two without meaning it.
- `docs/index.html` is a six-line redirect to `climate_health_catalogue.html`. GitHub Pages serves `index.html` at the site root, and without it the bare URL 404s. It holds no copy of the page, so it never goes stale.
- **Treat everything committed as public.** The repository is private today but is prepared to be opened, and opening it would expose every past commit as well as the current one. Internal material is therefore kept out of the repository altogether rather than merely out of `docs/`: the gaps register lives in `gaps.json`, the DOCX is built from it, and both are in `.gitignore`, along with `EDITING_GUIDE.pdf`. Before committing anything new, ask whether it could be read by a stranger — for this repository that is the only test that matters.

| Look and feel

The page follows the palette and typefaces of <https://mauroccm.github.io> — washi off-white `#FDFCF9`, ink `#3A3A3A`, pastel-red accent `#D96666`, Montserrat for headings and Noto Sans JP for text. Those live in the `:root` block of `build_web.py`; change them there, not in the generated HTML.

Two things are deliberately *not* taken from the personal site. The domain and access colours — health, climate, environment, reference, derived, and open/by-request/restricted — carry meaning, so they stay as they are rather than collapsing into the single accent. And the small uppercase labels stay monospace, because they mark structure rather than voice.

| Field standard

The 27 fields follow DCAT (the EU and Brazilian government standard for data catalogues) with four additions that matter for climate-health work and are not in DCAT: **analysis unit**, **latency**, **linkage key** and **known limitations**. Latency is the field that decides whether a dataset can support an operational decision. Linkage key is the field that makes the catalogue usable rather than merely descriptive.

Any organisation joining the catalogue fills in these fields for its own datasets and keeps its own data where it is. That is all the coordination required.

| Licence

Two kinds of material, licensed separately in `LICENCE`:

- **The build scripts** — `build_web.py`, `build_docx.js`, and the HTML, CSS and JavaScript they emit — under the **MIT Licence**.
- **The catalogue itself** — `catalogue.json`, the CSV register, the DOCX, the records on the published page, and this guide — under **CC BY 4.0**. Reuse it anywhere, including commercially, provided you attribute:

IPSP Climate and Health Metadata Catalogue, Institut Pasteur de São Paulo, <https://mauroccm.github.io/climate-health-catalogue/>

Copyright © 2026 Institut Pasteur de São Paulo and Mauro César Cafundó de Morais.

The distinction that matters: **this licence covers the descriptions, not the datasets they describe**. The catalogue is metadata. Each record's `licence` field carries the terms of the underlying data, and several of those are held by third parties or governed by a data-use agreement — nothing here grants any right to them.